Claude Multi-Account Fine Tuning

Claude Toolkit — User Manual

1. Orientation
2. Provider aliases (any LLM as a Claude Code alias)
   2.1 Demotion, and switching aliases across organizations
3. Local models through llmctl (and the validation gate)
4. Decision models (Jev) — the §11.4.279 layer
5. Other utilities
6. Project
7. Honest scope

Claude Toolkit — User Manual

This is the single entry point to the toolkit's documentation (Constitution §11.4.257: manual + guides + FAQ per feature, §11.4.212: everything reachable from the README). Each feature has a guide for doing the task and an FAQ for the questions people actually ask; the manual below indexes them in reading order.

1. Orientation

You want to…	Read
Install and run several Claude Code accounts on one host	README quick start, Fine-Tuning deep dive
Know where claude_toolkit sits in a constitution-governed project	README hub-engine section, placement diagram
See every command	README daily commands

2. Provider aliases (any LLM as a Claude Code alias)

- Guide: Provider Aliases User Guide
- FAQ: Provider FAQ
- Verification: Provider Verification Guide
- Kimi Code accounts: Kimi Accounts User Guide
- Diagrams: index (quota-limits-flow, kimi-family, enable-policy)

2.1 Demotion, and switching aliases across organizations

- `claude-providers sync` only REPORTS orphaned provider aliases; to demote them run `claude-providers sync --demote` (opt-in, v1.33.2).
- Each `claudeN` alias is a separate Anthropic organization. `claude-unify` shares `projects/` and `sessions/`, so resuming a session under an alias of another organization makes Claude Code print "can't use thinking from another organization": thinking-block signatures are organization-bound, so the CLI drops them and re-reads the whole session uncached (slower, more tokens). This cannot be avoided at API level for the unmodified session. The toolkit warns before launch (with the context size and a same-org alias to use instead); `CMA_SESSION_SANITIZE_ON_ORG_CHANGE=1` resumes a sanitized copy without thinking blocks (original kept); `claude-session sanitize-fork <config_dir> [sid]` does it by hand. Provider aliases are not Anthropic organizations and are not guarded.
- Opt-in Jev wiring of every supported agent home: `jev-setup.sh --base-homes` or `JEV_WIRE_AGENT_HOMES=1`; hosted decision endpoints stay SHADOW.

3. Local models through llmctl (and the validation gate)

- Quickstart: llmctl quickstart
- Guide: llmctl user guide — includes §5a, the §11.4.277 validation gate (`LLMCTL_VALIDATE_STRICT=1`, `llmctl require-validated`) and the §11.4.278 Vision QA relationship
- FAQ: llmctl FAQ
- Diagrams: detection, switch

4. Decision models (Jev) — the §11.4.279 layer

- Quickstart: Jev quickstart

- Guide: Jev user guide (gate, MCP server, stats, calibration, the HTTPS + bearer-key decide gateway, honest limitations)
- FAQ: Jev FAQ
- Diagrams: architecture, decide-gateway flow

5. Other utilities

- OpenCode Integration — share Skills + MCP + `CLAUDE.md`
- Session & Color — per-project session naming
- TOON Integration — token-efficient JSON
- Multi-host rollout

6. Project

- CHANGELOG — release history
- Diagram index — Mermaid sources and rendered SVGs

7. Honest scope

The manual indexes documentation that exists; it does not claim features beyond what the guides state. The decide-gateway transport (HTTPS + bearer key) is verified only against a hermetic TLS stand-in, not a real llmctl gateway (Jev guide, limitation 11). The toolkit forwards the llmctl validation gate's environment and honours its refusals but runs no Vision known-answer suite of its own.